

Received 21 June 2023, accepted 28 June 2023, date of publication 3 July 2023, date of current version 11 July 2023.

Digital Object Identifier 10.1109/ACCESS.2023.3291420

RESEARCH ARTICLE

Mitigating Cyber Attacks in LoRaWAN via Lightweight Secure Key Management Scheme

JUNAID QADIR^{1,2}, (Graduate Student Member, IEEE), ISMAIL BUTUN^{1,3}, (Member, IEEE),
PAOLO GASTALDO², ORAZIO AIELLO², (Senior Member, IEEE),
AND DANIELE D. CAVIGLIA², (Life Member, IEEE)

¹Division of Network and Systems Engineering, School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, 100 44, Stockholm, Sweden

²Department of Electrical, Electronic and Telecommunications Engineering and Naval Architecture (DITEN), University of Genoa, 16145 Genoa, Italy

³Department of Computer Engineering, Ostim Technical University, 06374 Ankara, Turkey

Corresponding authors: Junaid Qadir (junaidq@kth.se) and Ismail Butun (butun@kth.se)

This research was supported by KTH Royal Institute of Technology, Sweden.

ABSTRACT Owing to the geographically scattered end devices (*EDs*) in long-range wide area networks (LoRaWAN), devices that combat challenging cyber threats and attacks are of critical significance. In this perspective, LoRa Alliance[®] is continuously evolving the security of LoRaWAN and recently introduced a new version i.e., LoRaWAN 1.1x that is featured with security improvement. However, the wireless nature of LoRaWAN implementation still leaves it vulnerable to security breaches that compromise its integrity. Several problems have been pinpointed in the newer version such as one issue with key distribution in LoRaWAN 1.1 is that the keys are often pre-installed on the devices at the time of manufacturing. It can introduce security risks if the keys are not adequately protected or if the devices are compromised before they are deployed. In other words, the pre-installed keys may not be updated regularly, which can also introduce security risks. Thus, the keys need to be handled securely to maintain the security of the network and the over-the-air firmware updates feature could introduce new security challenges for the key distribution. This paper presents a key generation and distribution (KGD) mechanism that securely exchanges the root key between the *ED* and the application server (*AS*). The KGD protocol provides authentication by integrating Advanced Encryption Standard (AES-128) in addition to a secure hash function known as *Argon2*. The proposed protocol utilizes Elliptic-Curve Diffie-Hellman (ECDH) key exchange method that makes the protocol resilient to cyber threats. The ECDH algorithm exchanges the keys on the insecure channels and is, therefore, vulnerable to Man-in-the-Middle (MITM) attacks in the network. Therefore, to validate the key agreement and avoid adversaries, the KGD protocol considers the Elliptic Curve Digital Signature Algorithm (ECDSA) that authenticates and allows legitimate instances in the network. In last, a formal security analysis using the Scyther tool validates the security enhancement of the KGD protocol.

INDEX TERMS LoRa, LoRaWAN, security, key derivation, key exchanging, ECDH, security enhancement.

I. INTRODUCTION

Cyber attacks on connected devices and systems can potentially jeopardize the security and privacy in the low-power wide area network (LoRaWAN) communication protocol. As the LoRaWAN's proliferation continues to be widely increasing in the Internet of Things (IoT) applications [1], the risk of cyber attacks targeting the network's CIA triad

(i.e., confidentiality, integrity, and availability) is also increasing. LoRaWAN elucidates the security in the network and application layers [2]. On the one hand, the application layer ensures the confidentiality of the end-to-end packet exchange between the *ED* and itself. On the other hand, the network layer security defines the authenticity and integrity of the packet advancement from the *ED* and vice versa. Despite strong cryptographic encryption techniques, there are still void holes in the security of LoRaWAN. As such, a security threat has been pinpointed in [3] in which the

The associate editor coordinating the review of this manuscript and approving it for publication was Stefano Scanzio¹.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see <https://creativecommons.org/licenses/by-nc-nd/4.0/>

authors stressed the conflict of interest between the network server (*NS*) and the *AS*. If not handled appropriately, it would trigger cyber attacks across the entire network.

Numerous research works in the literature revealed that LoRaWAN is susceptible to a number of attacks; as such, the reply attack and its prevention is discussed in [4] and [5]. The bit flipping attack and its countermeasure is presented in [6] in which some parts of the cipher text has been changed without decryption. The possible remedy is to shuffle the octets in the frame payload could secure the communication between the *ED* and the *NS*. Furthermore, the LoRaWAN specification is also susceptible to denial of service (DoS) attacks. To put it into practice, the researchers in [7] identified three vulnerabilities that helps in executing DoS attack against the *ED* in the LoRaWAN network. In addition, another notorious issue that make the specification security insufficient, is uncovered in [8] in which the authors analyzed the weakness in a nonce (“a number that used once”). Nonce is used in the join request message and may pose security threats in LoRaWAN’s network if generated improperly. With disclosing such attacks, the LoRaWAN specification is therefore required to enhance the security consideration. Doing so, the LoRa Alliance® is persistently improving the security odyssey and released the latest version of the specifications. The newer version LoRaWAN v1.1.x [9] is featured with the advanced security considerations by adding several encryption keys (discussed in Section II) and a trust-based entity called the join server (*JS*). However, researchers have penetrated every sphere of the LoRaWAN specifications and unveiled that there are still loopholes that could precipitate possible cyber attacks [10], [11]. In view of this, the researchers like Hofmann et al. from the Deutsche Telekom [12] have stressed, the new architecture is still vulnerable to several security breaches and the *ED* could be manipulated if the attacker’s hands on the device’s firmware.

Security is getting advanced as the LoRaWAN v1.1.x orchestrates the application key (AppKey) and the network key (NwkKey) to fill the risk gaps in the previous version v1.0.x. However, scientific research has disclosed other security breaches in v1.1. One of the security risks is the use of unrecommended cryptographic modes such as ECB operation in the join accept message [9]. It is important to remark that v1.1 is prone to availability attacks, as an attacker can observe and interrupt the packet exchange via an RF jamming tool. The victim *ED* is therefore required to resend the join request message after a timeout, if prevented the response, the *ED* gets loose from the network. This attack creates a void hole in the network and thus diminishes the network’s availability. Several problems in [13] are differentiated into three categories such as minor, major, and critical attacks in which the re-keying issue is highlighted as a critical risk.

Cyberattacks on the LoRaWAN end may not be challenging, because it does have a number of vulnerabilities that might be exploited if the network’s encryption keys were compromised. In contrast to the previous version, the *ED* in the LoRaWAN v1.1.x holds two root keys which are further

used to generate and delegate the session keys. But, it does not come into resistance in cyberattacks, as an attacker can use brute-force and dictionary attacks to crack the root key. In practice, the root key cracking is employed in [14] in which Cesar et al. used a pair of two messages such as the join request and join accept messages with the message integrity code (MIC). Making such activities cumbersome and protecting the network from attacks, this paper proposes the key generation and distribution (KGD) algorithm that mitigates the root key attacks and securely delegates the session keys between the *ED* and the *JS*. The key summary of the proposed work is described below in detail.

A. PROBLEM STATEMENT

The root key identifiers (AppKey and NwkKey, as shown in Fig. 1) are the critical components of the security of a LoRaWAN network. It is used to derive the NwkSKey and AppSKey that are used to authenticate communication between the device, the network, and the *AS*. As such, it is important to ensure that the root keys are kept secure and are not compromised by an attacker. If one of them is compromised, it could potentially allow unauthorized devices to join the network, which could compromise the security and integrity of the network.

Several potential security vulnerabilities that could affect the root keys in the LoRaWAN network are including;

- **ED physical attacks:** The device deployed in the deployment region may unveil its root keys if an attacker is able to physically approach the device. Most of the devices lack a hardware security module (HSM); therefore, an attacker with physical access can extract the root keys from the device.
- **Network attack:** A potential attack may occur if the message is being transferred between the *ED* and the *NS*. An attacker can intercept communication and may try to extract the root keys from the message.
- **Weak key generation:** If the root keys are generated with improper technique or insufficient randomness, an attacker may consider the cryptanalysis of the *ED* and extract the root keys.
- **Weak key updating:** If the root keys are not updated frequently enough, an attacker may be able to exploit vulnerabilities in the key to gain access to the network.
- **Compromised server:** The *ED* shares the root keys with the *NS*. If the security confidentiality of the root keys at the server is breached, all information that is being exchanged between the *ED* and the *NS* is breached.

B. MOTIVATIONS

Addressing the root keys’ breaches is one of the significant fields in LoRaWAN communications. The researcher in [13] and [15] urges that an attacker can potentially exploit communication in LoRaWAN, if the root key are not periodically updated. Hence, the security of the network could be violated if the root keys are unveiled to the attacker as

it is used to generate the session keys. The research study in [16], [17], [18], and [19] proposed distinct approaches to generate and update the root keys. However, several challenges are confronted as these approaches are noncompliant with the tiny *EDs*. Since, *EDs* are equipped with low computing resources and are unable to compute heavy cryptography algorithms. In addition, such approaches select a non-standard randomness that is used to generate the root key. Weak randomness can jeopardize the root keys that is injected into the memory of the microcontroller in the *ED*.

This study is motivated by the aforementioned fact; therefore, we propose a new lightweight and authenticated secure root key management in LoRaWAN. Furthermore, this approach is put into practice on a testbed in the real world, and its effectiveness is evaluated.

C. CONTRIBUTIONS

Contributing to the conventional key generation schemes [18], [20], [21] in LoRaWAN, this study proposes the KGD mechanism that updates and securely exchanges the root keys in LoRa possible. The KGD protocol makes use of Diffie-Hellman (DH) key exchange in addition of elliptic curve (EC) cryptography which makes the protocol resilient against cryptanalysis and other cyberattacks. In essence, this paper lists the following key points.

- **Key generate:** This work proposes a novel root key-generating mechanism to make the network more resilient against cyberattacks. In the KGD mechanism, the primary root key is used to generate the root keys that is used for authentication and encryption. The KGD considers a cryptographically secure random number generator to generate the initialization vector (IV) and AES-128 bit in CBC mode.
- **Key distribute:** To update the shared keys e.g., NwkSKey and AppSKey in the node deployed in a scattered area, the KGD protocol uses Elliptic Curve Diffie-Hellman (ECDH) key exchange method, as the RSA protocol is more computationally demanding. The ECDH algorithm uses a smaller key length size and provides high security at the cost of lower energy consumption.
- **Key authenticate:** The proposed KGD protocol considers an elliptic curve digital signature algorithm (ECDSA) to authenticate communication between the *ED* and the *JS*. By using ECDSA, only the authorized node is allowed to communicate with the relevant *NS*.
- **Formal security analysis:** Unlike the lightweight key exchange approach in [22], we analyze the security vulnerability of the proposed KGD algorithm and provide formal security analysis using the Scyther tool focusing on the key exchange technique. We analyze, explore, and inspect the security features of the KGD protocol. However, there is no vulnerability found against the KGD protocol; which, in turn, clarifies the integrity, secrecy, and authentication of the proposed implementation.

The remaining parts of the article are sorted as follows. Section II discusses cryptographic theory and practice. Section III presents the related study. Section IV discusses the preliminary of the proposed work. The proposed work is explained in Section V. Section VI and VII explains security verification and hardware implementation. Finally, Section VIII shows the experimental results and the overall work is concluded in IX.

II. CRYPTOGRAPHIC THEORY AND PRACTICE IN LoRaWAN

Due to the low power and long packet transiting, LoRaWAN is gaining popularity in the IoT world. As the number of users increases every day, there are also increasing security breaches that could lead the network to considerable damage. Thus, researchers, scientists, and engineers have put their efforts into practice and investigated the new version of LoRaWAN which is in line with the existing standard. LoRaWAN V1.0.x is augmented by concatenating the new server called “*JS*” which orchestrates the join procedure in a more secure fashion. Contrary to this, the newer version is equipped with three different *NSs* i.e., home, serving, and forward that potentially tackle inevitable distance problems. As can be seen in Fig. 1, the addition of the *NSs* enables packet roaming from local, wide, and worldwide contexts. The *ED* in LoRaWAN adapts two methods such as; Over-the-Air-Activation (OTAA) and Activation-by-Personalization (ABP) to participate in the network. This paper merely explains the OTAA method as the ABP procedure is beyond the scope of this work. To establish the OTAA process, the *ED* needs to store prior data such as; the Join Extended Unique Identifier (JoinEUI) is a unique 64-bit identifier assigned to each *NS*. The Device Extended Unique Identifier (DevEUI) is a unique 64-bit identifier assigned to each end device. The Device Nonce (DevNonce) is a random 16-bit value generated by the end device and included in the join request packet sent to the *NS* during the join process. Device root keys i.e., AppKey and NwkKey both are hard-coded in the *ED* during fabrication. The AppKey is a unique AES-128 bit key shared between the *ED* and the *AS*. It is used to encrypt and decrypt the payload of application data messages, and to derive the AppSKey, which is used to secure the communication between the *ED* and the *AS*. The NwkKey is a unique AES-128 bit key that is shared between all *ED* and the *NS*. As, several *NSs* are involved in v1.1; therefore, NwkKey is used to generate the session key for each server, and specific lifetime keys for the *JS*. The detail of each session key is mentioned below:

Session Keys: In LoRaWAN, the *NS* annexes the Message Integrity Code (MIC) to all uplink and downlink messages. Each *ED* has a specific session key which is generated from NwkKey, JoinNonce, JoinEUI, and DevNonce which are further explained below.

Forwarding Network Session Integrity Key (FNwkSIntKey): Equation 1 shows the *FNwkSIntKey* which is a network session key assigned to each *ED* that is used to calculate the

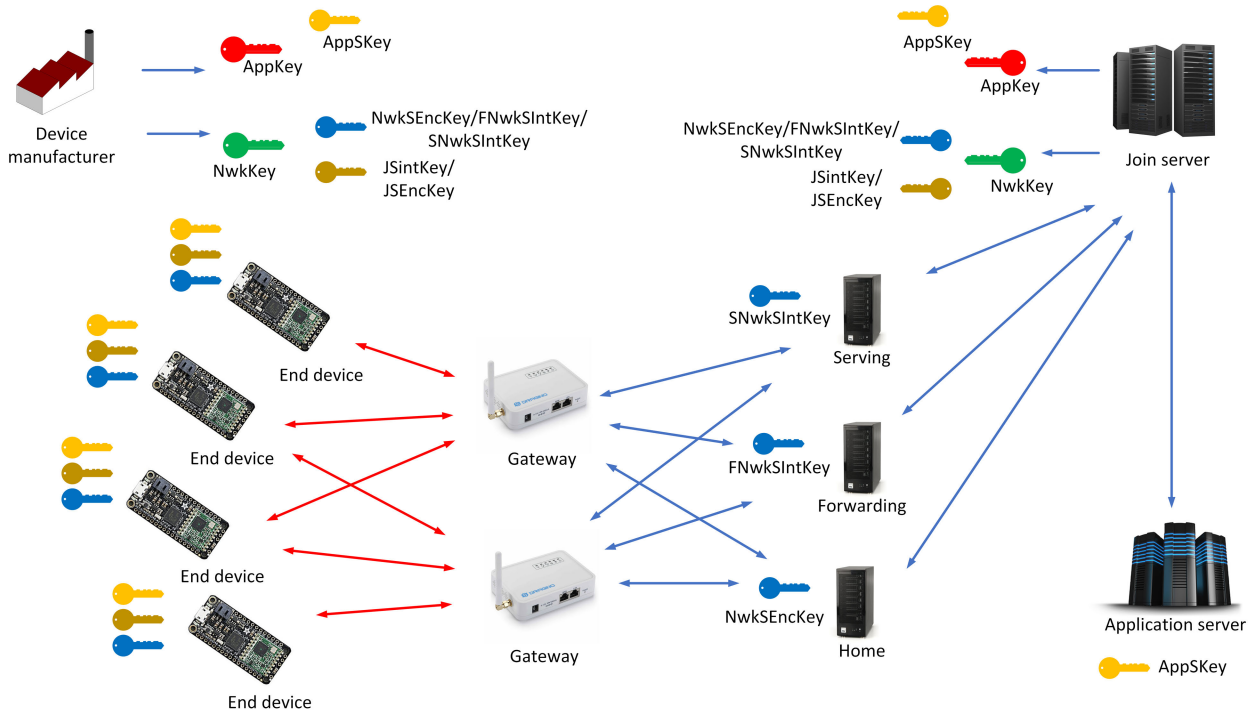


FIGURE 1. Network architecture of LoRaWAN.

MIC of all uplink transmission. This key can be shared with a roaming forwarding NS but it should not be made public because it could cause the network to experience issues [23].

$$FNwkSIntKey = aes128_enc(NwkKey, 0 \times 01 | JoinNonce | JoinEUI | DevNonce | pad16) \quad (1)$$

Serving Network Session Integrity Key (SNwkSIntKey): The SNwkSIntKey is given below in equation 2 which is used to calculate all downlink data messages in the network. Unlike the FNwkSIntKey, this key is considered private and should not be disclosed to the forwarding NS. The SNwkSIntKey is also used to compute MIC for half of uplink data messages and should be kept safe to avoid malicious activities.

$$SNwkSIntKey = aes128_enc(NwkKey, 0 \times 03 | JoinNonce | JoinEUI | DevNonce | pad16) \quad (2)$$

Network Session Encryption Key (NwkSEncKey): In the same fashion as the previous keys, the NwkSEncKey is a network session key and is specific to each ED. It is utilized to encrypt/decrypt uplink and downlink MAC commands that use port 0 or FOpt field as the way to exchange payload. In case the ED needs to connect to LoRaWAN v1.0.x, the NwkSEncKey is used to encrypt the MAC payload as well as compute MIC. Equation 3 shows the NwkSEncKey, that should not be disclosed to outsiders.

$$NwkSEncKey = aes128_enc(NwkKey, 0 \times 04 | JoinNonce | JoinEUI | DevNonce | pad16) \quad (3)$$

Lifetime Join Session Keys: As the inclusion of the JS in the new version, there are also two additional lifetime keys that are specifically used for rejoining the network when lost the connection. The JS integrity key (JSIntKey) is used for computing the MIC of rejoin request type 1 and related join accept responses. In addition, the JS encryption key (JSEncKey) is used to encrypt the response released by the rejoin request message.

$$JSIntKey = aes128_enc(NwkKey, 0 \times 06 | DevEUI | pad16) \quad (4)$$

$$JSEncKey = aes128_enc(NwkKey, 0 \times 05 | DevEUI | pad16) \quad (5)$$

Equation 4 and 5 show both keys that are only required in connecting the ED via the OTAA method. The ABP activation process is not obliged to store these keys in the device before activation.

III. RELATED WORK

This section discusses several key management techniques that have previously been published in the literature.

The authors in [24] discussed the vulnerabilities in the existing LoRaWAN's key exchanging method and proposed the Key distribution and updating mechanism (KDUM) to strengthen the network's security. The KDUM protocol makes use of updating the root and session keys to become the network more resilient against cyber-attacks. Furthermore, the authors argued that asymmetric algorithms for instance;

TABLE 1. Analysis of different approaches for key update in LoRaWAN.

Article	Algorithm's Approach(s)	Achievements	Flaws/Limitations	Year	Reference
M. Leent <i>et al.</i>	Proposed a key generation algorithm	The protocol possesses a good security framework that enhanced the existing LoRa end	The formal security verification is not considered	2017	[24]
E. Jayasuriya <i>et al.</i>	keys management method is proposed that is practically applied on the sensor node	Evaluates the network's efficiency, battery power consumption, and security strength	Disabling existing LoRaWAN feature may lose the quality of service	2021	[25]
Lin Tsai <i>et al.</i>	A server session key generation (S2KG) approach is presented to securely establish communication among different servers	The algorithm achieves good confidentiality, packet integrity, and mutual authentication while generating session keys	The algorithm is not considered on practical implementation, therefore, it may put an extra burden on node's memory	2020	[26]
Sanchez Iborra <i>et al.</i>	Proposed a scheme based on the DH key exchange method which is considered a convenient remedy	A feasible solution to enable the key exchange process	Compliant with the LoRaWAN v1.0.x and it does not cover the newer version of LoRaWAN v1.1	2018	[27]
Han and Wang	Proposed the Rabbit Cipher-based key management technique to enhance the complexity of cryptanalysis of the security keys	Outperforms in terms of less computing than the LoRaWAN key generated mechanism	No perfect forward secrecy is guaranteed	2018	[17]
Kim and Song	Considered a dual key-based approach for LoRaWAN and fixed the issue of key updates not receiving full support	Outperforms in terms of battery consumption and latency as compared to the original LoRaWAN implementation	This approach does not guarantee perfect forward secrecy, since the session keys are generated from the previous session	2017	[28]
Marlind and Butun	Public key over the air activation (PK-OTAA) is proposed to delegate the keys securely	It saves the join request message from external attacks	Consumes more battery power compared to the original LoRaWAN	2020	[29]
Chen <i>et al.</i>	This algorithm considered the Rabbit Cipher stream that takes a 128-bit secret and 64-bit initial vector as input	Achieves less good battery power management	Requires high computing efficiency	2021	[30]
Tan <i>et al.</i>	The permissioned-blockchain-based key management technique is proposed	Enhanced the authenticity and availability of LoRaWAN by keeping in view the low latency of processing	Hyperledger fabric is more energy-demanding, as LoRa nodes operate with limited battery power	2021	[31]
Hayati <i>et al.</i>	Proposed an approach that used the Photon-256 algorithm to produce a unique session key for secure communication in LoRaWAN	Achieved good security as it has validated via GNY logic and the protocol prover known as Scyther	Not fully compliant with the LoRaWAN implementation	2022	[16]

RSA is demanding high computational power, while IoT devices have low computational abilities. Therefore, they considered the DH algorithm to exchange the session key remotely. To configure the algorithm robust, two varieties i.e., discrete logarithms in a finite field (DH), and ECDH were available. However, they considered ECDH as it utilizes less number of bits and is high-security level. ECDH consumes less energy as compared to the DH and RSA algorithms, and hence the sensor node can be active for a long time. The KDUM protocol possesses a good security framework. However, the key limitations of the LoRaWAN ED are not addressed with this algorithm.

The conventional LoRaWAN implementation relies on security keys, as the root keys are hardcoded in the EDs at the time of the manufacturing process. The root keys are then used to create the session keys to secure communication. However, there might be security risks if the security keys are generated for a long duration. Therefore, the keys management method is proposed in [25] that is practically applied on the sensor node. The ECDH protocol with Micro-ECC setup is tested with various curves and evaluates the network's efficiency, battery power consumption, and security strength. The ECDH implementation is employed with the MCCI-LMIC library and disabled some extra features that cause

energy consumption. This protocol achieves good security with the cost of low flash memory consumption.

Lin Tsai et al. in [26] pointed out an issue with the existing LoRaWAN's key commissioning method. According to the authors, the network entity in LoRaWAN v1.0.x generates and utilizes the AES based key to authenticate communication and guarantee the integrity between *EDs* and *AS*. However, the communication of different servers is not defined in the specification. Therefore, a server session key generation (S2KG) approach is presented to securely establish communication among different servers. In the S2KG method, the *NS* utilizes symmetric key cryptography, while the *JS* makes use of elliptic curve cryptography (ECC) to enhance the security of the communication. The S2KG method achieves good confidentiality, packet integrity, and mutual authentication while generating session keys.

In [27], the authors discussed security vulnerabilities in the LoRaWAN v1.0 and proposed a key management scheme. The proposed scheme is based on the DH key exchange method which is considered a convenient remedy because of the less computational overhead and use of fewer cryptographic bits. This scheme is then compared with the IKEv2 [32], DTLS [33], and EDHOC [34], which is found a feasible solution to enable the key exchange process. However, the work is solely compliant with the LoRaWAN v1.0.x and it does not cover the newer version of LoRaWAN v1.1. Therefore, this work needs to be updated considering the security requirement of version v1.1.

Han and Wang in [17] argued that the security of LoRaWAN v1.1 comes up with the basic security requirements. The current key generation method in LoRaWAN uses AES-ECB which is considered not a viable choice as it is vulnerable to pattern analysis. In other words, the AES-ECB consumes more battery power from the sensor node. Therefore, they proposed the Rabbit Cipher-based key management technique to enhance the complexity of cryptanalysis of the security keys. The rabbit is a stream cipher that is used in the two-step key derivation function to obtain the pseudo-random number generator. Simulation results reveal that the proposed method achieves good performance in terms of less computing than the LoRaWAN key generated mechanism. Also, it provides a high number of randomness which makes the protocol more robust against cryptanalysis.

Kim and Song in [28], stressed the security loopholes in the existing LoRaWAN's key update and session key generation. They considered a dual key-based approach for LoRaWAN and fixed the issue of key updates not receiving full support. In this approach, the *NwkKey* and *AppKey* are used to generate the initial join request process. Then the session keys that are derived from the previous join process are considered for the second join process. The process solves the issue of the update which is not taken into consideration in the existing LoRaWAN implementation. Furthermore, this approach allows each layer to establish a unique session key so that the layers can operate separately. The real-world experiment

demonstrates that the proposed approach outperforms in terms of battery consumption and latency as compared to the original LoRaWAN implementation. However, this approach does not guarantee perfect forward secrecy, since the session keys are generated from the previous session.

Another interesting research carried out by Marind and Butun in [29]. According to the authors, the *EDs* in LoRaWAN come up with reconfigured root keys. And the session keys are generated using the root key, so there is a potential of key disclosing if the *ED* is susceptible to security breaches. Changing the root key in the *ED* is cumbersome since it requires a physical presence to the device. They provide an alternative remedy whereas the root key can assign remotely using public key cryptography. This method is also known as the public key over the air activation (PK-OTAA) since it uses elliptic curve cryptography (ECC). The main reason for using the ECC is because of the shorter key length size. Furthermore, the root key is generated using ECDH that exchanges the secret value. This protocol provides a significant security improvement to the LoRaWAN joining procedure. However, it consumes more battery power compared to the original LoRaWAN.

The authors in [30] argued that the new release of LoRaWAN i.e., v1.1 has significantly improved the security framework; however, the process of key delegation is still vague and needs to be defined explicitly. They proposed a new key management setup to tackle the issues with key updating, generation, and exchanging from the node through the server. The proposed setup considered the Rabbit Cipher stream that takes a 128-bit secret and 64-bit initial vector as input. Simulation reveals that the Rabbit Cipher algorithm is much faster than the AES-ECB mode and consumes less battery power.

To enhance the authenticity and availability of LoRaWAN, the permissioned-blockchain-based key management technique is proposed in [31]. The new architecture of the LoRaWAN is considered whereas the *NS* and the *JS* are controlled through blockchain and the key exchange process is accomplished using ECDH which provides high security with minimal resource usage. To assess the achievement, this approach is compared with the ChirpStack *NS*, and the formal security prover known as AVISPA is used to verify the security. Simulation results reveal that the proposed approach outperforms in terms of packet processing latency.

Hayati et al. in [16] proposed an approach that used the Photon-256 algorithm to produce a unique session key for secure communication in LoRaWAN. This approach first considers the initialization stage for the session key at the *ED*. And the contents of the session key are collected at the *JS*. In addition, a set of key pairs i.e., *NwkSKey* and *AppSKey* are created at the *ED* and the *NS*. This approach is verified and validated via GNY logic and the protocol prover known as Scyther. The proposed approach achieves a good security framework and economical solution. However, it is

not fully compliant with the LoRaWAN implementation. The key features of comparison is analyzed in table 1.

IV. PRELIMINARY

In this section, we first discuss the key generation procedure and then the possible way of key sharing between the *ED* and *NS* using the *DH* method. At last, we discuss our proposed key distribution with the ECDH key exchange method.

A. KGD: KEY GENERATION

The first step that makes up the KGD implementation into practice is the key generation process. The KGD method uses the NIST-approved encryption algorithm named AES counter-based deterministic random bit generator (AES-CTR-DRBG), which is a type of random number generator based on a block cipher, and uses a counter mode to generate random numbers [35]. The strength of AES-CTR-DRBG depends on the block cipher used, and AES-128 is considered to be a secure choice. This algorithm turns a fixed-length block cipher into a stream cipher. It generates a keystream by encrypting a counter and then XORing the keystream with plaintext. The AES-CTR provides a high degree of randomness that makes the system more difficult to predict as it was verified in [36]. Also, the research in [16] utilized this method for obtaining random bits in a key updating procedure in LoRaWAN. However, researchers in [37] and [38] stressed the vulnerability found in the AES-CTR. A practical attack is possible on this technique and could obtain the internal input information. Cohny et al. suggested that this attack could be avoided by updating the input seeds frequently. But, frequent update of the input seeds is not an optimal solution, and the LoRa devices are equipped with limited power and processing unit.

B. KEY DISTRIBUTION WITH DH

DH is the earliest mathematical model that is used to exchange cryptographic keys securely over a public/insecure channel. In the context of LoRa session keys, the devices (*ED_{ith}*) and the *JS* (*J_{sr}*) generate a private key also known as a secret key. To employ DH key exchanging algorithm, all the *ED_{ith}* and *J_{sr}* agree on a prime *P* and generator point *G*. The term *G* must be a primitive root of *P* and must consider lower than *P*. Using these parameters, *ED_{ith}* and *J_{sr}* exchange a 128-bit long shared key as shown in the equations below. *ED_{ith}* calculates public key (*ED_{pub}*) from its secret key *ED_{sec}*.

$$ED_{pub} = G^{ED_{sec}}(modP) \quad (6)$$

J_{sr} calculates public key *JS_{pub}* from the secret key *JS_{sec}* in the same fashion as the *ED_{ith}*.

$$JS_{pub} = G^{JS_{sec}}(modP) \quad (7)$$

ED_{ith} computes the shared key *ED_{sh}* from its *ED_{sec}* and *JS_{pub}*.

$$ED_{sh} = JS_{pub}^{ED_{sec}}(modP) \quad (8)$$

TABLE 2. Notations used in the proposed approach.

Notation	Definition
ED_{pub}	Public key of the <i>ED</i>
ED_{sec}	Secret/private key of the <i>ED</i>
G	Generator point
JS_{pub}	Public key of the <i>JS</i>
JS_{sec}	Secret/private key of the <i>JS</i>
ED_{sh}	Shared key of the <i>ED</i>
JS_{sh}	Shared key of the <i>JS</i>
$\mathbb{I}$	Identity element
h	Hashed message
k	Random bit generated
ED_{ith}	The number of <i>EDs</i>

J_{sr} computes the shared key *JS_{sh}* from its *JS_{sec}* and *ED_{pub}*.

$$JS_{sh} = ED_{pub}^{JS_{sec}}(modP) \quad (9)$$

The shared keys i.e., *ED_{sh}* == *JS_{sh}* work in the same fashion as the current root keys do in the LoRa end. Using shared keys, several session keys i.e., *FNwkSIntKey*, *SNwkSIntKey*, *NwkSEncKey*, and *AppSKey* can be generated to put an extra security layer on the payload being exchanged between the *ED_{ith}* and multiple servers. The key notations used in these equations are depicted in the table. 2.

Elliptic Curve: An elliptic curve is an algebraic curve that consists of points (x,y) over finite fields $\mathbb{F}_p$ (where p is a prime number) and is generated by the following cubic degree

$$Ax^3 + Bx^2y + Cxy^2 + Dy^3 + Ex^2 + Fxy + Gy^2 + Hx + Iy + J = 0, \quad (10)$$

however, cryptography uses the simplest, as such the Weierstrass form [39] which is given as:

$$Y^2 = x^3 + ax + b(modp), \quad (11)$$

elliptic curves such as NIST curve secp256k1 are widely used in bitcoin's public key cryptography defined by the standard for efficient cryptography (SEC) from the Certicom research [40]. All curves must make sure if they satisfy the condition given in the equation below

$$4a^3 - 27b^2 \neq 0. \quad (12)$$

V. PROPOSED WORK

In the proposed work, we used a well-vetted, cryptographic-grade random number generator library to seed the DRBG, such as Argon2, a password hashing winner in 2015 [41]. Argon2 itself is a key derivation function that is designed to be secure against various attacks such as brute-force and dictionary attacks. It is not a DRBG itself, but it can be used to derive a cryptographic key that can be used as a seed for a DRBG. Argon2 can help AES-CTR DRBG in terms of security by using it to derive a key for the AES-CTR DRBG from a password or a secret. This would provide an

additional layer of security because an attacker would need to know both the password and the key in order to predict the output of the DRBG. In other words, it can help AES-CTR DRBG in terms of security by using it to derive a nonce for the AES-CTR DRBG. Using a nonce together with a key in AES-CTR DRBG increases the security of the generated random bits. The Argon2 lists several input parameters as shown in Fig. 2, whereas T is the time cost that controls the number of iterations, M shows the amount of memory being used. The parameters P and H show parallelism that controls the number of threads, and hash length, consequently. The last two parameters P_s show the input password to be taken, while S is the salt. All these steps can make the Argon2 algorithm more suitable for a 32-bit microprocessor. DRBG takes all these parameters as input and passes through AES-CTR mode with the static key S_k to obtain both the NwkKey and AppKey at both the JS and end node. The JoinNonce is a random variable that is generated at the JS while DevNonce is at the end node. Now, we have root secrets keys as shown in the red and green colors in Fig. 3 which are then used to generate session keys. But updating them at the ED is challenging because it is often deployed in a scattered remote area where physical access is not possible. As in Figure 3, the ED sends a join request message (highlighted in red) through the gateway to the JS. The join request message contains relevant information about the ED and JS, such as unique identifiers and DevNonce. Upon receiving the join request message, the JS verifies the authenticity of the ED and performs other security checks. If the ED is authenticated successfully, the JS sends a join accept response message (highlighted in blue) to the ED through the gateway. This message contains the session keys that are used to establish a secure communication channel between the ED and the AS. The session keys are delegated to both the ED and the AS to ensure secure communication between them.

In LoRaWAN, the root keys are used to generate several session keys that are distributed among several servers and the ED. The purpose of each session key is discussed in section II. To exchange these session keys remotely, a key exchange algorithm is required that distributes the session key to the LoRa end. Several key exchange methods are available in the literature; however, this work considers Diffie-Hellman (DH) key exchange method. Other key exchanging methods like RSA, which consumes more power and is high computational demanding. To use the DH method, two variants such as DH with discrete logarithms in a finite field (discussed in section IV (B)) and ECDH could be utilized. ECDH provides a convenient and secure method of key exchanging between the ED_{ith} and J_{sr} in LoRaWAN standard. As discussed the key generation process in Section IV(A), we now exchange the key in the given KGD solution. To take the ECDH into consideration, the ED_{ith} and J_{sr} must choose a private key that will use in the public key of each other. To generate the shared key, the ECDH uses the curve given in the equation 11 that helps in determining the shared keys. To perform the ECDH, the ED_{ith} and J_{sr} must agree on the prime (p) number

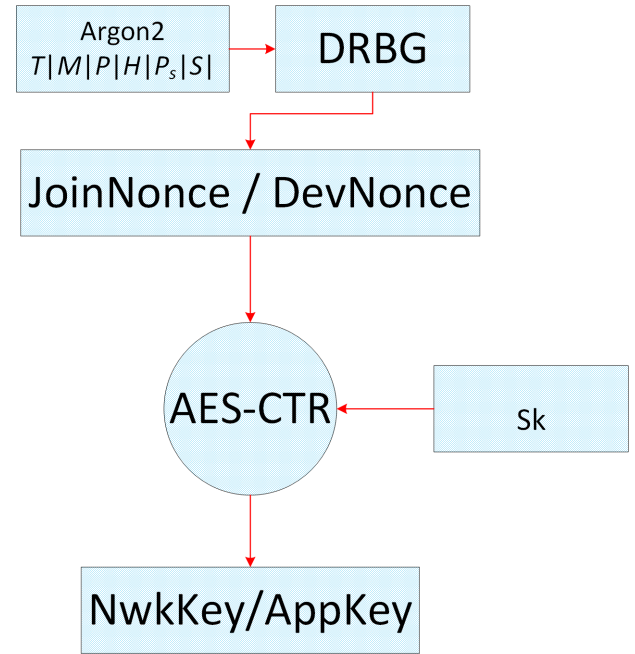


FIGURE 2. Root keys generation in the JS/ED.

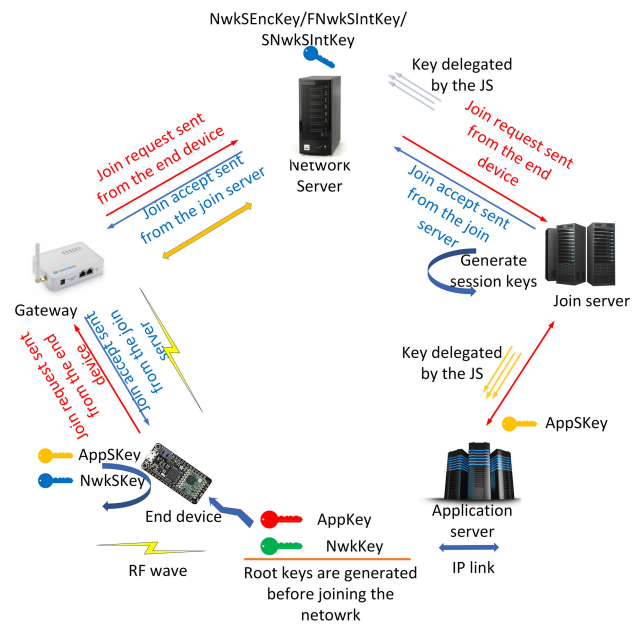


FIGURE 3. Root and session key delegation at the JS/ED.

generator (G) point publicly. Both parties must satisfy the requirements given in equation 12; otherwise, the points may lie outside of the chosen curve. The key exchanging of LoRaWAN ED and server using ECDH follows as:

- 1) If E is the elliptic curve over a finite field with a prime number such as $E\mathbb{F}_p$, then the key exchange takes place as
- 2) The ED_{ith} and J_{sr} chooses a point P in $E\mathbb{F}_p$

- 3) The ED_{ith} selects a secret key ED_{sec} and calculates the public key ED_{pub} . In mathematical terms,

$$ED_{pub} = ED_{sec} \cdot P \in E\mathbb{F}p, \quad (13)$$

- 4) The ED_{ith} shares ED_{pub} with the J_{sr}
 5) The J_{sr} chooses a secret key JS_{sec} and computes the public key JS_{pub} . In mathematical terms,

$$JS_{pub} = JS_{sec} \cdot P \in E\mathbb{F}p, \quad (14)$$

- 6) The J_{sr} shares JS_{pub} with the ED_{ith}
 7) The ED_{ith} computes the shared key received from the J_{sr} as

$$\begin{aligned} ED_{sh} &= ED_{sec} \cdot JS_{pub} \in E\mathbb{F}p, \\ &= ED_{sec} \cdot (JS_{sec} \cdot P) \in E\mathbb{F}p, \\ &= ED_{sec} \cdot JS_{sec} \cdot P \in E\mathbb{F}p, \end{aligned} \quad (15)$$

- 8) The J_{sr} computes the shared key received from the ED_{ith} as

$$\begin{aligned} JS_{sh} &= JS_{sec} \cdot ED_{pub} \in E\mathbb{F}p, \\ &= JS_{sec} \cdot (ED_{sec} \cdot P) \in E\mathbb{F}p, \\ &= JS_{sec} \cdot ED_{sec} \cdot P \in E\mathbb{F}p. \end{aligned} \quad (16)$$

- 9) Both the ED_{ith} and J_{sr} can communicate securely using the shared key $ED_{sh} == JS_{sh}$.

VI. SECURITY VERIFICATION

This section presents the verification in two methods. Firstly, we present the informal verification of the proposed approach, using the elliptic curve digital signature algorithm (ECDSA). Secondly, we provide formal verification using the Scyther verification tool. The details and discussions of both verification methods are presented below.

A. KGD: KEY AUTHENTICATION

The proposed KGD implementation considers the key authentication phase, as we discussed the key generation and distribution between the ED_{ith} and J_{sr} over an insecure channel using the ECDH method. However, the authentication of the key being exchanged is important as there is a possibility that the ED_{ith} may share the key with an adversary or vulnerable J_{sr} , and vice versa. To cope with the problem, the authenticated key exchange is a key solution that could potentially avoid key revealing breaches. The proposed KGD uses the elliptic curve digital signature algorithm (ECDSA) since it comes up with the same foundation as in ECDH. The ECDSA consumes less energy of the node as compared to other rivals such as RSA, ElGamal, and DSA; because it takes a smaller key length size and low computation power. ECDSA algorithm is based on the mathematics of equation 11. Likewise the ECDH, the ED_{ith} and J_{sr} are required to disclose some information publicly.

In ECDSA, a private key is a randomly generated number, which is used to generate a public key through a mathematical operation. The public key is then used to generate a digital

signature for a message. The ECDSA algorithm follows two parts such as signature generation and verification. To generate a signature, it is important to first compute a hash of the message. In the proposed LoRa KGD solution, we compute the hash function of the public key of the ED_{ith} and J_{sr} using the SHA-256 hashing function. It is worth noting the hash function results in a larger bit length; however, we kept only the leftmost bits to compute the signature. For the sake of simplicity, we present a simple example that we considered as a key authentication process in the proposed KGD solution using the ECDSA algorithm. The simple method is as follows:

Key Generation:

- 1) The ED_{ith} generate(s) a secret/private key ED_{sec} from the random or deterministic random number generation such as [1, n-1].
- 2) In this case, the ED_{sec} is for instance the number “7”.
- 3) Using the ED_{sec} , the public key ED_{pub} is calculated as

$$ED_{pub} = ED_{sec} \cdot G \quad (17)$$

where, G is the generator point on the curve having x and y coordinates for instance $(x, y) = (3, 7)$

- 4) For example the ED_{pub} is a point on the curve having coordinates $(x_1, y_1) = (14, 28)$

Message Hashing:

- 1) Using the cryptography library in python “Hashlib” [42], we compute the hash function for the message being transmitted to the J_{sr}
- 2) In this example, we hashed the message using SHA3-256

$$h = \text{hashlib.sha3_256}(\text{message}) \quad (18)$$

where hash “h” is for instance $h = 12345$

Signature Generation:

- 1) The ED_{ith} generate(s) a random number for instance $k = 3$
- 2) The signature in ECDSA uses two values (r,s), where the x_1 coordinate is used to compute the value r

$$r = x_1 \text{ modulo } n \quad (19)$$

- 3) The variable n denotes the infinity points on the curve. Here we suppose its value is $n = 29$
- 4) The x_1 coordinate is already given such as $x_1 = 14$; therefore, equation 19 can be written as

$$\begin{aligned} r &= x_1 \text{ modulo } n \\ r &= 14 \text{ modulo } 29 \\ r &= 14 \end{aligned} \quad (20)$$

- 5) It is also worth mentioning that if r is equal to zero then k must be regenerated. The value of r is computed now we define the value of s as follows below

$$\begin{aligned} s &= k^{-1}(h + ED_{sec} * r) \text{ modulo } n \\ s &= 3^{-1}(12345 + 7 * 14) \text{ modulo } 29 \end{aligned} \quad (21)$$

- 6) The modular inverse of k with respect to n is calculated as is $k * k^{-1} \equiv 1 \pmod{n}$ or equivalently $k * k^{-1} \pmod{n} = 1$

$$\begin{aligned}
 3 * 0 &\equiv 0 \pmod{29} \\
 3 * 1 &\equiv 3 \pmod{29} \\
 3 * 2 &\equiv 6 \pmod{29} \\
 3 * 3 &\equiv 9 \pmod{29} \\
 &\dots \dots \dots \dots \dots \\
 &\dots \dots \dots \dots \dots \\
 3 * 8 &\equiv 24 \equiv 24 \pmod{29} \\
 3 * 9 &\equiv 27 \equiv 27 \pmod{29} \\
 3 * 10 &\equiv 30 \equiv \boxed{1} \pmod{29} \\
 3 * 11 &\equiv 33 \equiv 4 \pmod{29} \\
 &\dots \dots \dots \dots \dots \\
 &\dots \dots \dots \dots \dots \\
 3 * 27 &\equiv 81 \equiv 23 \pmod{29} \\
 3 * 28 &\equiv 84 \equiv 26 \pmod{29}
 \end{aligned}$$

- 7) The modular inverse of k with respect to n is “10”, but to check the signature validity, let’s take an incorrect integer “9”. So, from equation 21

$$\begin{aligned}
 s &= 3^{-1}(12345 + 7 * 14) \pmod{29} \\
 s &= 9(12345 + 98) \pmod{29} \\
 s &= 9(12443) \pmod{29} \\
 \boxed{s} &= \boxed{26}
 \end{aligned} \tag{22}$$

Equations 20 and 22 show the signature $(r,s) = (14, 26)$ is calculated that will further append to the public key of the J_{sr} and, vice versa.

Signature Verification:

- 1) After the signature generation at the ED_{ith} , the J_{sr} then verify the signature and check the authenticity of the message being exchanged.
- 2) To verify the signature, the J_{sr} takes the following parameters as input such as; the signed message, the signature $(r,s) = (14,26)$ and the $ED_{pub}=(x_1, y_1)=(14,28)$ of the ED_{ith} .
- 3) The J_{sr} authenticates the message using the modular inverse of s with respect to n . The modular inverse is computed in the same fashion as in equation 22.

$$\begin{aligned}
 w &= s^{-1} \pmod{n} \\
 w &= 26^{-1} \pmod{29} \\
 w &= 13
 \end{aligned} \tag{23}$$

- 4) The J_{sr} then calculate u_1 and u_2 to recover the random points used in the signature generation step.

$$\begin{aligned}
 u_1 &= (h * w) \pmod{n} \\
 u_1 &= (12345 * 13) \pmod{29} \\
 \boxed{u_1} &= \boxed{8}
 \end{aligned}$$

$$\begin{aligned}
 u_2 &= (r * w) \pmod{n} \\
 u_2 &= (14 * 13) \pmod{29} \\
 \boxed{u_2} &= \boxed{5}
 \end{aligned} \tag{24}$$

- 5) The J_{sr} uses the ED_{pub} of the ED_{ith} , and the G point. By scalar multiplication, we get

$$\begin{aligned}
 (x_2, y_2) &= u_1 * G + u_2 * ED_{pub}, \\
 (x_2, y_2) &= 8 * (3, 7) + 5 * (14, 28), \\
 (x_2, y_2) &= (24, 56) + (70, 140), \\
 (x_2, y_2) &= (112, 196).
 \end{aligned} \tag{25}$$

- 6) When the ED_{ith} send(s) a message towards the J_{sr} , it will first check if the message being sent is original or not. If the value of r is equal to x i.e., $r \equiv x$, then it means the received signature is valid. Otherwise, it will consider as tempering or integrity has been attacked. Hence, with the KGD mechanism, the LoRaWAN key authenticity is proven where $r = 14 \neq x_2 = 112$.

$$r = x_2 \pmod{n} \tag{26}$$

Figure. 4 provides a visual representation of the key authentication process. The process involves the ED signing the message using the JS_{pub} and sending it to the JS . The JS then uses the JS_{sec} to authenticate the message received from the ED . Similarly, the ED authenticates the messages received from the JS using the JS_{sec} . This two-way authentication process ensures the security and integrity of the communication between the ED and JS .

B. FORMAL SECURITY VERIFICATION

Verification of the proposed KGD algorithm is one of the most important steps that have been evaluated by the automated prover tool Scyther [43], [44], [45], developed by researchers in CISPA: the Helmholtz Center for Information and Security [46]. It is worth noting that several other tools like Avispa [47], ProfVerif [48], and Tamarin [49] are available in the literature. However, this work considers Scyther as it is easy to use, lightweight and has an attractive GUI that uses the Security Protocol Description Language (SPDL) programming language to characterize the role of security analysis. The KGD protocol is scrutinized and validated using the Scyther tool as shown in Fig. 5. The Scyther tool verified and passed several performance tests that show the KGD algorithm is secured against several attacks. The non-injective agreement (Niagree) parameter is verified that reflects both the ED_{ith} and JS agree to exchange the packets. If they did not exist within the range, then the Scyther tool shows “Fail” instead of “Ok”, and graphically visualizes possible attacks. Furthermore, the feature of non-injective synchronization (Nisynch) indicates that the content exchanged between both entities is similar and no integrity attack has been devised. The parameter “Alive” shows the two parties are available and ensures the existence for content exchanging between ED_{ith} and the JS . The parameter

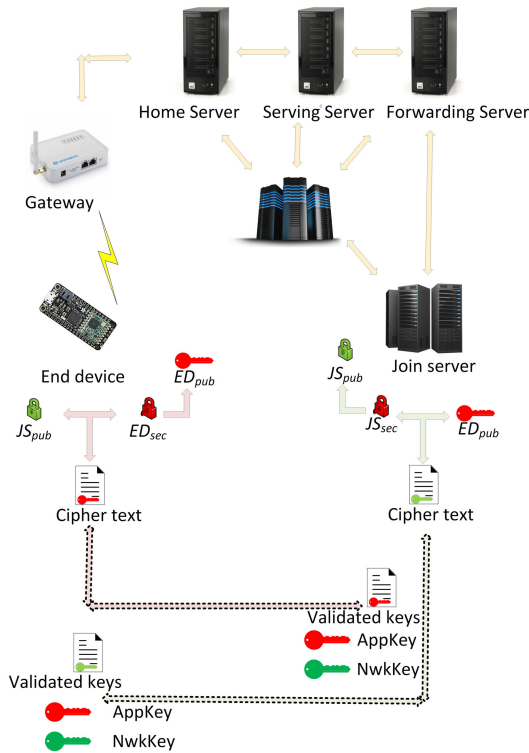


FIGURE 4. Authentication of the keys using ECDSA in the JS/ED.

Scyther results : characterize

Claim	Status	Comments	Patterns
LoRaWANKGD EndDevice LoRaWANKGD,EndDevice1	Reachable	Ok Verified	Exactly 1 trace pattern. 1 trace pattern
JoinServer LoRaWANKGD,JoinServer1	Reachable	Ok Verified	Exactly 1 trace pattern. 1 trace pattern

Done.

Scyther results : verify

Claim	Status	Comments
LoRaWANKGD EndDevice LoRaWANKGD,ED1	Secret data	Ok Verified No attacks.
LoRaWANKGD,ED2	Niagree	Ok Verified No attacks.
LoRaWANKGD,ED3	Nisynch	Ok Verified No attacks.
LoRaWANKGD,ED4	Alive k	Ok Verified No attacks.
JoinServer LoRaWANKGD,JS	Secret rdata	Ok Verified No attacks.
LoRaWANKGD,JS2	Niagree	Ok Verified No attacks.
LoRaWANKGD,JS3	Nisynch	Ok Verified No attacks.
LoRaWANKGD,JS4	Alive	Ok Verified No attacks.

Done.

FIGURE 5. The KGD protocol validation using Scyther tool.

“Secret” verifies the secrecy of the communication and indicates that no activity is being attacked.

VII. HARDWARE IMPLEMENTATION

The section discusses the practical implementation of the KGD algorithm with the LoRaWAN testbed. For convenient

performance, we considered the adafruit RFM95 radio chip that is packaged with the SX1276 [50] LoRa transceiver. The LoRa transceiver is then connected with the Raspberry Pi (RPi) 4 model B which makes a complete LoRa node as used in [51]. This transceiver uses a few spread spectrum versions, such as SF7-SF12, to enable ultra-long-range communication. Furthermore, it provides good sensitivity with consuming very little energy. The LoRaWAN gateway acts as a bridge, that forwards data from the *ED* to the *NS* and vice versa. This paper uses the PicoCell SX1308 [52] LoRa concentrator from the Semtech corporation. The PicoCell is connected with the RPi 3 model B, as shown in Fig. 6. The gateway uses a LoRa packet forwarder [53] that forwards the packet using IP/UDP link, shown in Fig. 7. In this paper, we used the server from the TheThingsNetwork (TTN)¹ that provides a convenient solution using built-in integrations such as AWS, Azure, and MQTT to process the data come from the *ED*.

VIII. PERFORMANCE EVALUATION

This section elucidates the evaluation of the performance of the KGD solution, which is implemented on a Raspberry Pi 4 Model B in conjunction with the RFM95 LoRa module. In order to assess the LoRa capability, we disabled several peripheral features of the Raspberry Pi, such as HDMI, Ethernet, and external ports, except for the GPIO pins. This was done to minimize any potential interference and to allocate more system resources toward the LoRa module’s operation, thereby providing a more accurate evaluation of its performance. To evaluate the performance, we determined the total energy consumption of the Raspberry Pi when integrated with the RFM95 module. We established a connection between the PicoScope 2203 [54] and our experimental setup, and proceeded to acquire measurements while compiling our Python script. Figure 8 depicts the total energy and power consumption of the Raspberry Pi and RFM95 module when applying the KGD algorithm by utilizing the secure shell (SSH) protocol. The energy consumption (highlights in blue) obtained from the testbed exhibits a clear linear plot, indicating a consistent increase in power consumption (highlights in orange) over time. This linear relationship reflects the integral power consumption of the Raspberry Pi attached with the RFM95 when applying the KGD algorithm. To assess an accurate measurement, a total of 11,733 samples were collected using the Picoscope, ensuring a comprehensive analysis of energy and power consumption. By analyzing the figure, we can gain valuable insights into the efficiency of the testbed when executing the KGD algorithm.

A. PERFECT FORWARD AND BACKWARD SECRECY

The perfect forward and backward secrecy depends on the breach of the root keys either at the *ED* or *JS*. By hands-on the NwkKey or AppKey, an intruder can potentially disclose the secrecy, which in turn, poses a high risk to the entire network.

¹<https://eu1.cloud.thethings.network/console/>

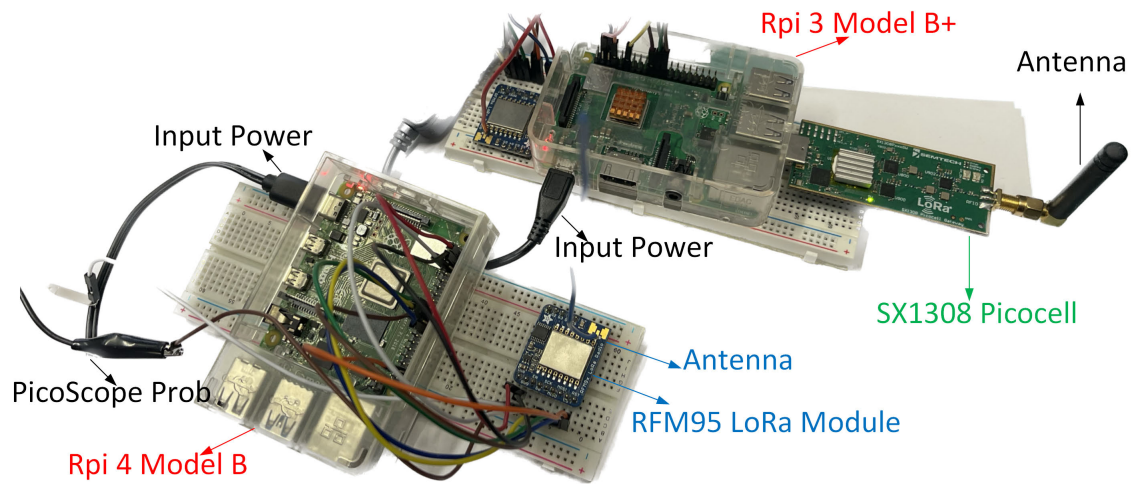


FIGURE 6. The testbed used in the experiment.

```

pi@raspberrypi:~/lorahackster$ cd lorahackster/picoGW_packet_forwarder/lor_pkt_fwd/
pi@raspberrypi:~/lorahackster/picoGW_packet_forwarder/lor_pkt_fwd$ ./lor_pkt_fwd
*** Packet Forwarder for LoRa PicoCell Gateway ***
Version: 0.1.0
*** Lora concentrator HAL library version info ***
Version: 0.2.3;
*** RCU FW version for LoRa PicoCell Gateway ***
Version: 0x010A0006
***
INFO: Little endian host
INFO: Found global configuration file global_conf.json, parsing it
INFO: global_conf.json does contain a JSON object named SX1301_conf, parsing SX1301 parameters
INFO: lorawan_public 1, clksrc 1
INFO: antenna_gain 0 dBi
INFO: Configuring TX LUT with 16 indexes
INFO: radio 0 enabled (type SX1257), center frequency 867500000, RSSI offset -164.000000, tx enabled 1
INFO: radio 1 enabled (type SX1257), center frequency 868500000, RSSI offset -164.000000, tx enabled 0
INFO: Lora multi-SF channel 0> radio 1, IF -400000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 1> radio 1, IF -200000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 2> radio 1, IF 0 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 3> radio 0, IF -400000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 4> radio 0, IF -200000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 5> radio 0, IF 0 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 6> radio 0, IF 200000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 7> radio 0, IF 400000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora std channel> radio 1, IF -200000 Hz, 250000 Hz bw, SF 7
INFO: FSK channel> radio 1, IF 300000 Hz, 125000 Hz bw, 50000 bps datarate
INFO: global_conf.json does contain a JSON object named gateway_conf, parsing gateway parameters
INFO: gateway MAC address is configured to 383438344C004100
INFO: server hostname or IP address is configured to "eu1.cloud.thethings.network"
INFO: upstream port is configured to "1700"
INFO: downstream port is configured to "1700"
INFO: downstream keep-alive interval is configured to 10 seconds
INFO: statistics display interval is configured to 30 seconds
INFO: upstream PUSH_DATA time-out is configured to 100 ms
INFO: packets received with a valid CRC will be forwarded

```

FIGURE 7. The raspberry pi terminal shows the PicoCell SX1308 receives and transmits packets.

To mitigate such risks, the proposed scheme employs periodic updates of the root keys, which are generated through the use of time T and input password P_s parameters of Argon2 in the key generation process. Consequently, even if one or both root keys are compromised, the intruder is unable to predict future keys or retrieve previous ones. This renders the network more resilient against potential attacks, since new keys are introduced periodically to enhance security.

B. STATISTICAL RANDOMNESS TEST

In this section, the proposed scheme is validated through a randomness test, which is an important aspect of evaluating the security of cryptographic schemes. This test helps to ensure that the generated keys are truly random and unpredictable, and therefore suitable for use in secure

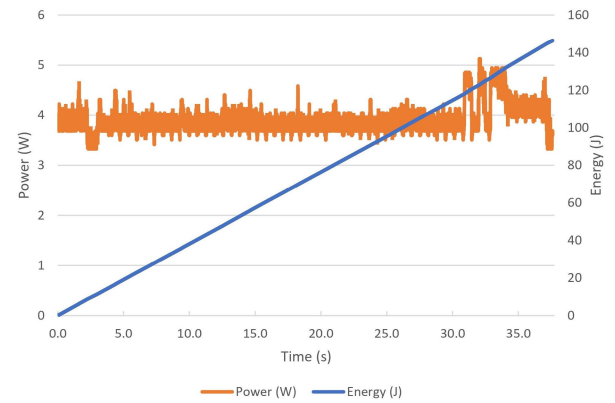


FIGURE 8. Analysis of energy and power consumption of the Raspberry Pi connected to the RFM95 LoRa module.

TABLE 3. NIST statistical test [57] suit for the uniformity of P-value.

No	Statistical Test	Proportion	P-Value	Assessment
1	Frequency	10/10	0.534146	Passed
2	BlockFrequency	10/10	0.739918	Passed
3	CumulativesSums	10/10	0.911413	Passed
4	CumulativesSums (a)	10/10	0.911413	Passed
5	CumulativesSums (b)	10/10	0.911413	Passed
6	Runs	10/10	0.122325	Passed
7	LongestRun	10/10	0.213309	Passed
8	Rank	10/10	0.911413	Passed
9	FFT	10/10	0.739918	Passed
10	Serial (a)	9/10	0.350485	Passed
11	Serial (b)	9/10	0.534146	Passed
11	LinearComplexity	10/10	0.739918	Passed

communication protocols. Therefore, to evaluate the statistical randomness of the proposed approach, we conducted two tests for the randomness that complied with the requirements of the National Institute of Standards and Technology (NIST) [55] and Diehard [56], respectively. Due to the limited

TABLE 4. Table of acronyms.

Acronym	Expansion
AES	Advance encryption standard
AppKey	Application key
AppSkey	Application session key
DH	Diffie Hellman
ECC	Elliptic curve cryptography
ECDH	Elliptic curve diffie hellman
FNwkSIntKey	Forwarding network session integrity key
HSM	Hardware security module
IoT	Internet-of-Things
IV	Initialization vector
KDUM	Key distribution and updating mechanism
KGD	Key generation and distribution
LoRaWAN	Low power wide area network
MITM	Man-in-the-Middle
MIC	Message integrity code
NekKey	Netowk key
NwkSEncKey	Networks session encryption key
NwkSKey	Network session key
PK-OTAA	Public key over the air activation
RSA	Rivest Shamir Adleman
SNwkSIntKey	Serving network session integrity key

computing capacity of the 32-bit microprocessor, we tested the proposed scheme on Ubuntu 22.04.2 LTS, with an Intel[®] Core[™] i5-10500 CPU @ 3.10 GHz x 12 and 8.0 GiB of RAM. We stored the input values in a transcendental number (.pi) file from NIST's suite and called the iteration 40196 times, whereas each iteration generates Additionally, we executed the Diehard test to ensure that all tests were passed and that no anomalies were detected. As demonstrated in table 3, the key generated by the proposed algorithm satisfied with the NIST's recommended randomness tests.

IX. CONCLUSION

LoRaWAN is an emerging technology that has opened a large pool of innovation in the field of IoT. It has diminished the constraints in long-distance packet advancement by keeping in view very less energy consumption. LoRaWAN incorporates advanced security features for both the network and the application layers by employing several cryptographic keys. The ED, however residing on the edge of the network is a primary target for cyber attackers. This paper presents a remedy called the KGD algorithm that mitigates cyberattacks in the light of secure key management. The KGD algorithm is accomplished in three steps. At first, it generates the secret keys with a cryptographically secured deterministic random bit generator method. The generated keys are then exchanged between the ED and JS using the ECDH key exchanging algorithm. To check if the keys are exchanged with the authentic entities, a key authentication process such as the ECDSA algorithm is considered to verify if the keys

were exchanged with the right parties. Finally, a protocol verification is executed using the Scyther protocol prover that shows the proposed KGD algorithm is secured against cyberattacks and possesses mutual authentication, integrity, availability, and perfect forward secrecy.

Future work may consider malware attacks between the JS and the AS as the LoRaWAN specification has not yet disclosed the security consideration between these two entities. Using efficient Machine Learning (ML) algorithm might be feasible to detect such attacks.

REFERENCES

- [1] J. Qadir, I. Butun, R. Lagerstrom, P. Gastaldo, and D. D. Caviglia, "Towards smart sensing systems: A new approach to environmental monitoring systems by using LoRaWAN," in *Proc. IEEE Zooming Innov. Consum. Technol. Conf. (ZINC)*, May 2022, pp. 176–181.
- [2] LoRaWAN Security. Accessed: Feb. 16, 2023. [Online]. Available: https://lora-alliance.org/wp-content/uploads/2020/11/lorawan_security_whitepaper.pdf
- [3] M. Ralambotiana, "Key management with a trusted third party using LoRaWAN protocol?: A study case for E2E security," Ph.D. dissertation, KTH Roy. Inst. Technol., Sweden, 2018.
- [4] J. Kim and J. Song, "A simple and efficient replay attack prevention scheme for LoRaWAN," in *Proc. the 7th Int. Conf. Commun. Netw. Secur.*, Nov. 2017, pp. 32–36.
- [5] W.-J. Sung, H.-G. Ahn, J.-B. Kim, and S.-G. Choi, "Protecting end-device from replay attack on LoRaWAN," in *Proc. 20th Int. Conf. Adv. Commun. Technol. (ICACT)*, Feb. 2018, p. 1.
- [6] J. Lee, D. Hwang, J. Park, and K.-H. Kim, "Risk analysis and countermeasure for bit-flipping attack in LoRaWAN," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 549–551.
- [7] E. van Es, H. Vranken, and A. Hommersom, "Denial-of-service attacks on LoRaWAN," in *Proc. 13th Int. Conf. Availability, Rel. Secur.*, Aug. 2018, pp. 1–6.
- [8] S. Zulian, "Security threat analysis and countermeasures for LoRaWAN join procedure," Ph.D. dissertation, Università degli Studi di Padova, Italy, 2016.
- [9] LoRaWAN Specification V1.1. Accessed: Feb. 16, 2023. [Online]. Available: https://lora-alliance.org/resource_hub/lorawan-specification-v1-1/
- [10] F. Hessel, L. Almon, and M. Hollick, "LoRaWAN security: An evolvable survey on vulnerabilities, attacks and their systematic mitigation," *ACM Trans. Sensor Netw.*, vol. 18, no. 4, pp. 1–55, Nov. 2022.
- [11] J. Qadir, I. Butun, P. Gastaldo, and D. D. Caviglia, "Review of security vulnerabilities in LoRaWAN," in *Proc. Int. Conf. Appl. Electron. Pervading Ind., Environ. Soc. Cham, Switzerland: Springer*, 2022, pp. 248–254.
- [12] Y. S. E. A. P. Hofmann. *Comparison and Analysis of Security Aspects of LoRaWAN and NB-IoT*. Accessed: Feb. 19, 2023. [Online]. Available: <https://iot.telekom.com/en/downloads/security-aspects-lorawan-nb-iot>
- [13] I. Butun, N. Pereira, and M. Gidlund, "Security risk analysis of LoRaWAN and future directions," *Future Internet*, vol. 11, no. 1, p. 3, Dec. 2018.
- [14] C. Cerrudo and E. Martinez, "LoRaWAN networks susceptible to hacking: Common cyber security problems, how to detect and prevent them," Tech. Rep., 2020.
- [15] I. Butun, N. Pereira, and M. Gidlund, "Analysis of LoRaWAN v1.1 security," in *Proc. 4th ACM MobiHoc Workshop Exp. Design Implement. Smart Objects, Jun.* 2018, pp. 1–6.
- [16] N. Hayati, K. Ramli, S. Windarta, and M. Suryanegara, "A novel secure root key updating scheme for LoRaWANs based on CTR_AES DRBG 128," *IEEE Access*, vol. 10, pp. 18807–18819, 2022.
- [17] J. Han and J. Wang, "An enhanced key management scheme for LoRaWAN," *Cryptography*, vol. 2, no. 4, p. 34, Nov. 2018.
- [18] T. C. M. Dönmez and E. Nigussie, "Key management through delegation for LoRaWAN based healthcare monitoring systems," in *Proc. 13th Int. Symp. Med. Inf. Commun. Technol. (ISMICT)*, May 2019, pp. 1–6.
- [19] J. Xing, L. Hou, K. Zhang, and K. Zheng, "An improved secure key management scheme for LoRa system," in *Proc. IEEE 19th Int. Conf. Commun. Technol. (ICCT)*, Oct. 2019, pp. 296–301.
- [20] X. Chen, J. Wang, and L. Wang, "A fast session key generation scheme for LoRaWAN," in *Proc. Austral. New Zealand Control Conf. (ANZCC)*, Nov. 2019, pp. 63–66.

- [21] S. A. A. Hakeem, S. M. A. El-Kader, and H. Kim, "A key management protocol based on the hash chain key generation for securing LoRaWAN networks," *Sensors*, vol. 21, no. 17, p. 5838, Aug. 2021.
- [22] M. Vučinić, G. Selander, J. P. Mattsson, and T. Watteyne, "Lightweight authenticated key exchange with EDHOC," *Computer*, vol. 55, no. 4, pp. 94–100, Apr. 2022.
- [23] J. J. Barriga and S. G. Yoo, "Securing end-node to gateway communication in LoRaWAN with a lightweight security protocol," *IEEE Access*, vol. 10, pp. 96672–96694, 2022.
- [24] M. Leent, "An improved key distribution and updating mechanism for low power wide area networks (LPWAN)," Ph.D. dissertation, Universiteit Leiden, The Netherlands, 2017.
- [25] E. N. Jayasuriya, "ECDH based key management for LoRaWAN considering sensor node limitations," Ph.D. dissertation, Univ. Colombo, Sri Lanka, 2021.
- [26] K.-L. Tsai, F.-Y. Leu, L.-L. Hung, and C.-Y. Ko, "Secure session key generation method for LoRaWAN servers," *IEEE Access*, vol. 8, pp. 54631–54640, 2020.
- [27] R. Sanchez-Iborra, J. Sánchez-Gómez, S. Pérez, P. Fernández, J. Santa, J. Hernández-Ramos, and A. Skarmeta, "Enhancing LoRaWAN security through a lightweight and authenticated key management approach," *Sensors*, vol. 18, no. 6, p. 1833, Jun. 2018.
- [28] J. Kim and J. Song, "A dual key-based activation scheme for secure LoRaWAN," *Wireless Commun. Mobile Comput.*, vol. 2017, pp. 1–12, Nov. 2017.
- [29] F. Märlind and I. Butun, "Activation of LoRaWAN end devices by using public key cryptography," in *Proc. 4th Cyber Secur. Netw. Conf. (CSNet)*, Oct. 2020, pp. 1–8.
- [30] X. Chen, M. Lech, and L. Wang, "A complete key management scheme for LoRaWAN v1.1," *Sensors*, vol. 21, no. 9, p. 2962, Apr. 2021.
- [31] M. Tan, D. Sun, and X. Li, "A secure and efficient blockchain-based key management scheme for LoRaWAN," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Mar. 2021, pp. 1–7.
- [32] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, and T. Kivinen, *Internet Key Exchange Protocol Version 2 (IKEv2)*, document RFC7296, 2014.
- [33] E. Rescorla and N. Modadugu, *Datagram Transport Layer Security*, document RFC4347, 2006.
- [34] G. Selander, J. Mattsson, and F. Palombini, *Ephemeral Diffie-Hellman Over CoSe (EDHOC)*, document draftietf-lake-edhoc-19, Internet Engineering Task Force, Internet-Draft, Feb. 2023.
- [35] E. Barker, *Recommendation for Key Management Part 1: General*, Standard NIST SP 800-57pt1r4, National Institute of Standards and Technology, Jan. 2016, doi: [10.6028/NIST.SP.800-57pt1r4](https://doi.org/10.6028/NIST.SP.800-57pt1r4).
- [36] M. Aljohani, I. Ahmad, M. Basher, and M. O. Allassafi, "Performance analysis of cryptographic pseudorandom number generators," *IEEE Access*, vol. 7, pp. 39794–39805, 2019.
- [37] V. T. Hoang and Y. Shen, "Security analysis of NIST CTR-DRBG," in *Advances in Cryptology—CRYPTO 2020*. Santa Barbara, CA, USA: Springer, Aug. 2020, pp. 218–247.
- [38] J. Woodage and D. Shumow, "An analysis of NIST SP 800-90A," in *Advances in Cryptology—EUROCRYPT 2019*. Darmstadt, Germany: Springer, May 2019, pp. 151–180.
- [39] J. W. S. Cassels, "Review: Joseph H. Silverman, The arithmetic of elliptic curves," *Bull. Amer. Math. Soc.*, 17, no. 1, pp. 148–149, Jul. 1987.
- [40] D. R. Brown, "Sec 2: Recommended elliptic curve domain parameters," Standards Efficient Cryptogr., Tech. Rep., 2010.
- [41] J. Wetzel, "Open sesame: The password hashing competition and Argon2," 2016, *arXiv:1602.03097*.
- [42] G. P. Smith, *Secure Hashes and Message Digests*. Accessed: Feb. 8, 2023. [Online]. Available: <https://docs.python.org/3/library/hashlib.html>
- [43] K. Inoue, K. Wada, and Y. Ito, "Effective application of Paro: Seal type robots for disabled people in according to ideas of occupational therapists," in *Computers Helping People with Special Needs*. Linz, Austria: Springer, Jul. 2008, pp. 1321–1324.
- [44] S. Mauw and C. Cremers, *Operational Semantics and Verification of Security Protocols* (Information Security and Cryptography). Springer, 2012.
- [45] F. A. Putra, K. Ramli, N. Hayati, and T. S. Gunawan, "PURA-SCIS protocol: A novel solution for cloud-based information sharing protection for sectoral organizations," *Symmetry*, vol. 13, no. 12, p. 2347, Dec. 2021.
- [46] C. Cremers, *The Scyther Tool*. Accessed: Feb. 8, 2023. [Online]. Available: <https://people.cispa.io/cas.cremers/scyther/index.html>
- [47] A. Armando, D. Basin, Y. Boichut, Y. Chevalier, L. Compagna, J. Cuellar, P. H. Drielsma, P. C. Heám, O. Kouchnarenko, J. Mantovani, S. Mödersheim, D. von Oheimb, M. Rusinowitch, J. Santiago, M. Turuani, L. Viganò, and L. Vigneron, "The AVISPA tool for the automated validation of internet security protocols and applications," in *Computer Aided Verification*. Edinburgh, U.K.: Springer, Jul. 2005, pp. 281–285.
- [48] B. Blanchet, "An efficient cryptographic protocol verifier based on prolog rules," in *Proc. CSFW*, vol. 1, 2001, pp. 82–96.
- [49] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN prover for the symbolic analysis of security protocols," in *Computer Aided Verification*. Saint Petersburg, Russia: Springer, Jul. 2013, pp. 696–701.
- [50] Semtech, *Semtech Corporation SX1276*. Accessed: Feb. 8, 2023. [Online]. Available: <https://www.semtech.com/products/wireless-RF/lora-core/sx1276>
- [51] A. Mohamed, F. Wang, I. Butun, J. Qadir, R. Lagerström, P. Galstado, and D. D. Caviglia, "Enhancing cyber security of LoRaWAN gateways under adversarial attacks," *Sensors*, vol. 22, no. 9, p. 3498, May 2022.
- [52] *SX1308P915GW. Semtech Corporation SX1276*. Accessed: Feb. 8, 2023. [Online]. Available: <https://www.semtech.com/products/wireless-RF/lora-core/sx1308p915gw>
- [53] LoRa-Net, *Picocell LoRaWAN Gateway Forwarder*. Accessed: Feb. 8, 2023. [Online]. Available: https://github.com/Lora-net/picoGW_packet_forwarder
- [54] *Picoscope 2203*. Accessed: Feb. 8, 2023. [Online]. Available: <https://www.picotech.com/download/manuals/ps2203.en-4.pdf>
- [55] S.-J. Kim, K. Umeno, and A. Hasegawa, "Corrections of the NIST statistical test suite for randomness," 2004, *arXiv:nlin/0401040*.
- [56] M. M. Alani, "Testing randomness in ciphertext of block-ciphers using diehard tests," *Int. J. Comput. Sci. Netw. Secur.*, vol. 10, no. 4, pp. 53–57, 2010.
- [57] A. Rukhin et al., "A statistical test suite for random and pseudorandom number generators for cryptographic applications," U.S. Dept. Commerce, Technol. Admin., Nat. Inst. Standards Technol., 2001, vol. 22.



JUNAID QADIR (Graduate Student Member, IEEE) is currently pursuing the Ph.D. degree with the Department of Electrical, Electronic and Telecommunications Engineering, and Naval Architecture (DITEN), University of Genoa, Italy. He was a Visiting Ph.D. Student with the Division of Network and Systems Engineering, KTH Royal Institute of Technology, Stockholm, Sweden. Prior to this, he was a Research Collaborator with the Department of Signal Theory, Communications and Telematics Engineering, University of Valladolid, Spain. He has published his research work in prestigious journals, such as *IEEE Access*, *Energies* (MDPI), *Journal of Intelligent and Fuzzy Systems*, and several conferences. His current research interests include the Internet of Things, LoRaWAN, network security, underwater wireless sensor networks, wireless sensor networks, and mobile edge computing (MEC). He is a Reviewer with a number of prestigious international publishers, such as *IEEE Access*, *IEEE SENSORS JOURNAL*, *International Journal of Distributed Sensor Networks* (IJDSN), *Journal of King Saud University* (Elsevier), *Computer Methods and Programs in Biomedicine* (Elsevier), *Heliyon*, *IEEE International Wireless Communications and Mobile Computing Conference (IEEE IWCMC 2019)*, *Network Modeling and Analysis in Health Informatics and Bioinformatics*, and *Acta Acustica united with Acustica the Journal of the European Acoustics Association (EAA)*.



ISMAIL BUTUN (Member, IEEE) received the B.Sc. and M.Sc. degrees in electrical and electronics engineering from Hacettepe University and the M.Sc. and Ph.D. degrees in electrical engineering from the University of South Florida, in 2009 and 2013, respectively. From 2014 to 2017, he was an Assistant Professor with the Department of Mechatronics Engineering, Bursa Technical University, and the Department of Computer Engineering, Abdullah

Gul University. From 2016 to 2021, he was a Postdoctoral Fellow for various universities, such as the University of Delaware, Mid Sweden University, and the Chalmers University of Technology. Since 2021, he has been with the School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology. He has more than 50 publications in peer-reviewed scientific international journals and conference proceedings, along with an H-index of 20. His research interests include computer networks, wireless communications, WSNs, the IoT, cyber-physical systems, cryptography, network security, and intrusion detection. He is a well-recognized academic reviewer by IEEE, ACM, and Springer, who served more than 50 various scientific journals and conferences in the review process of more than 200 articles. He is an Editor of *IEEE Access*, *Sensors* (MDPI), *Journal of Sensors* (Hindawi), and *Nature* (Springer).



PAOLO GASTALDO received the Laurea degree in electronic engineering and the Ph.D. degree in space sciences and engineering from the University of Genoa, Italy, in 2004. He is currently an Associate Professor with DITEN, University of Genoa, where he teaches computer architecture and sensors for electronic systems. His main research interests include embedded machine learning, computational intelligence, embedded systems for advanced signal interpretation in

robotics and prosthetics, and cryptography.



ORAZIO AIELLO (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees (cum laude) from the University of Catania, Italy, in 2005 and 2008, respectively, the M.Sc. degree (cum laude) from Scuola Superiore di Catania, Italy, in 2009, and the Ph.D. degree from Politecnico di Torino, Italy, in 2013. He was a Research Fellow with Politecnico di Torino, in a joint project with FIAT-Chrysler Automobiles, Turin. He was a Mixed Signal IC Designer and an EMC Consultant with

STMicroelectronics, Castelletto, Italy, from 2008 to 2009, and NXP-Semiconductors, Nijmegen, The Netherlands, in 2014. He was a Visiting Ph.D. Student with Monash University, Melbourne, Australia, in 2013, and a Visiting Fellow with the University of Sydney and The University of New South Wales, Sydney, Australia, in 2015 and 2016, respectively. Since 2015, he has been with the Green IC Group, National University of Singapore, where he has also been a Marie Skłodowska-Curie Individual and a Global Fellow. He is currently a tenure-track Assistant Professor with the University of Genoa, Italy. His main research interests include energy-efficient analog-mixed signal circuits and sensor interfaces. He is a member of the IEEE CASS Microlearning AdHoc Committee. He is/was a Technical Program Committee Member of a number of conferences, such as NORCAS and APCCAS. He is an Associate Editor of *IET Electronics Letters*. He serves as a regular reviewer for several IEEE journals.



DANIELE D. CAVIGLIA (Life Member, IEEE) received the degree in electronic engineering and the degree in computer engineering from the University of Genoa, Genoa, Italy, in 1980 and 1982, respectively. In 1983, he joined the Institute of Electrical Engineering, University of Genoa, as an Assistant Professor of electronics, where he has been with the Department of Biophysical and Electronic Engineering (DIBE), since 1984, as an Associate Professor, in 1992, and as a Full Pro-

fessor, in 2000. From 2002 to 2008, he was also the Director of DIBE, University of Genoa. He is currently with the Department of Electrical, Electronics and Telecommunications Engineering and Naval Architecture (DITEN), University of Genoa, teaching courses in electronic systems for telecommunications, electronic devices and circuits, and microelectronics. He is also active in the development of innovative solutions for environmental monitoring for civil protection and security applications. His research interests include the design of electronic circuits and systems for telecommunications, electronic equipment for health and safety, and energy harvesting techniques for the Internet of Things (IoT) applications.

...